

Maritime CargoService: Sprint 0

Davide Chirichella, Gabriele Doti, Daniele
Maccagnan

Ingegneria dei Sistemi Software, A.A. 2025/2026
Alma Mater Studiorum · Università di Bologna

June 15, 2026

1. Introduction

Una compagnia di trasporto marittimo di container (d'ora in poi *la committente*) intende automatizzare le operazioni di carico dei container nella stiva della nave (d'ora in poi **hold**). A tal fine prevede di impiegare un robot a guida differenziale (Differential Drive Robot, d'ora in poi **cargorobot**).

L'obiettivo dello Sprint 0 è formalizzare i requisiti forniti dalla committente in modo preciso e non ambiguo, costruire un primo modello dei macro-componenti del sistema distinguendo ciò che è fornito da ciò che deve essere sviluppato, evidenziare il *core business*, motivare la scelta del linguaggio di modellazione e definire un primo insieme di piani di test funzionali.

Ogni scelta è strettamente ancorata ai requisiti: non si anticipano né decisioni progettuali né scelte implementative, che verranno affrontate negli sprint successivi.

1.1. Glossario

TERMINE	DEFINIZIONE
hold	Area rettangolare piatta che costituisce la stiva della nave. Contiene slot1–slot4 (immagazzinamento definitivo), slot5 (transito) e l'IOPort (punto di ingresso/uscita).
slot1–slot4	Quattro aree distinte della hold, ciascuna capace di contenere un container. Costituiscono la destinazione finale di ogni container caricato.
slot5	Area speciale in cui il cargorobot deposita temporaneamente un container prima della collocazione definitiva, in attesa dell'etichettatura.
IOPort	Dispositivo di ingresso/uscita dotato di: pushbutton (richieste di carico), display (stato e messaggi) e sonar (rilevazione presenza container).
customer	Attore esterno che interagisce fisicamente con l'IOPort per richiedere il carico e per depositare il container nell'area del sonar.
cargorobot	Robot a guida differenziale (DDR) fornito dalla committente, responsabile della movimentazione fisica dei container nella hold.
cargoservice	Servizio software principale da costruire. Orchestra il ciclo di carico: ricezione richieste, verifica precondizioni, guida del cargorobot, etichettatura, aggiornamento display, controllo LED.
marker device	Dispositivo fisico in slot5 che appone un codice a barre al container e segnala il completamento dell'etichettatura.
sonar	Sensore associato all'IOPort. Rileva la presenza di un container quando misura $D < D_{\text{FREE}}/2$ per almeno 3 secondi.

D_{FREE}	Distanza soglia del sonar. Se $D > D_{\text{FREE}}$ per almeno 3 secondi si presuppone un malfunzionamento.
LED	Indicatore luminoso che lampeggia durante lo stato <i>engaged</i> .
engaged	Stato in cui una richiesta è stata accettata e l'operazione è in corso.
disengaged	Stato normale di attesa: nessuna operazione in corso.
Out of service	Stato anomalo ($D > D_{\text{FREE}}$ per ≥ 3 s): cargoservice risponde <i>retrylater</i> a ogni richiesta.

2. Requirements

2.1. Requisiti funzionali

- Il servizio principale da costruire è **cargoservice**.
- Il customer invia a cargoservice una richiesta di carico tramite il pushbutton dell'IOPort.
- Se l'IOPort è occupata o il sistema è *Out of service*, cargoservice risponde **retrylater**.
- Se tutti gli slot1–slot4 sono occupati, cargoservice **rifiuta** la richiesta.
- Altrimenti, cargoservice entra in stato *engaged*, riserva uno slot libero e restituisce al customer il nome dello slot riservato.
- Fintanto che il sistema è *engaged*, un **LED lampeggia**.
- Dopo l'accettazione, il customer ha un tempo prefissato (es. 30 s) per depositare il container nell'area del sonar.
- Se il customer non deposita entro il timeout, il sistema torna *disengaged*.
- Quando la presenza del container è confermata dal sonar, cargoservice comanda al cargorobot di spostare il container **dall'IOPort a slot5**.
- Il marker device etichetta il container in slot5 e **segnala il completamento**.
- cargoservice comanda al cargorobot di spostare il container **da slot5 allo slot riservato**.
- Il display mostra in ogni momento lo **stato corrente della hold**.
- Il display mostra il messaggio **“Service working”** durante il normale funzionamento.
- Se il sonar misura $D > D_{\text{FREE}}$ per ≥ 3 s, il sistema passa *Out of service* e mostra **“Out of service”** sul display.
- Il sonar rileva la presenza di un container quando misura $D < D_{\text{FREE}}/2$ per ≥ 3 s.

2.2. Requisiti non funzionali

- La hold è un'area rettangolare piatta con slot1–slot4, slot5 e IOPort.
- L'IOPort è dotata di pushbutton, display e sonar.
- Il cargorobot è un DDR fornito dalla committente.
- Il tempo massimo di attesa per il deposito è un parametro prefissato (es. 30 s).
- La soglia temporale per le rilevazioni del sonar è di almeno 3 secondi.

2.3. Domande aperte alla committente

Domanda al committente. D1. Posizione dell'IOPort nella mappa. I requisiti indicano che il cargorobot sposta il container *dall'IOPort a slot5*, lasciando intendere che l'IOPort sia una cella praticabile. Come si colloca nella griglia?

Domanda al committente. D2. Trasporto fisico dei container. I requisiti non specificano come il cargorobot trasporti fisicamente i container. Il robot li spinge, li raccoglie con un meccanismo, oppure la gestione meccanica è trasparente al software?

Domanda al committente. D3. Area di copertura del sonar. La *sensor area* coincide con l'area dell'IOPort o è una cella adiacente? Il chiarimento è essenziale per definire il comportamento di rilevazione e movimentazione.

Domanda al committente. D4. Interfaccia del pushbutton. Il pushbutton è un dispositivo fisico dedicato oppure può essere realizzato tramite un'interfaccia software?

3. Requirement analysis

3.1. Core business

Il **core business** del sistema è la gestione del ciclo di carico di un container. La sequenza principale ricavata dai requisiti è:

1. Il customer preme il pushbutton dell'IOPort.
2. cargoservice verifica le precondizioni (stato sistema, IOPort, disponibilità slot).
3. Se soddisfatte: stato *engaged*, prenotazione slot, notifica al customer.
4. Il customer deposita il container nell'area del sonar entro il timeout.
5. cargoservice comanda al cargorobot di spostare il container da IOPort a slot5.
6. Il marker device etichetta il container e segnala il completamento.
7. cargoservice comanda al cargorobot di spostare il container da slot5 allo slot riservato; il sistema torna *disengaged*.

3.2. Macro-componenti e natura software

COMPONENTE	RUOLO	STATO
cargoservice	Orchestratore principale. Natura reattiva (richieste) e proattiva (comandi).	Da sviluppare
cargorobot	Robot DDR per la movimentazione fisica. Simulatore e interfaccia forniti.	Fornito
IOPort	Interfaccia con il customer (pushbutton + display). Fisico fornito; sw da sviluppare.	Fisico fornito sw da sviluppare

sonar	Sensore di distanza. Fisico fornito; driver sw da sviluppare.	Fisico fornito sw da sviluppare
marker device	Etichettatura in slot5. Fisico fornito; sw da sviluppare.	Fisico fornito sw da sviluppare
LED	Indicatore stato <i>engaged</i> . Fisico fornito; sw da sviluppare.	Fisico fornito sw da sviluppare
hold	Struttura dati per lo stato della stiva (occupazione slot). Passivo.	Da sviluppare

3.3. Motivazione dell'uso del linguaggio QAK

Il linguaggio QAK non è assunto come vincolo a priori; la sua adozione è motivata da tre evidenze ricavate dai requisiti:

- **Natura reattiva e proattiva di cargoservice:** il servizio risponde a stimoli esterni (richieste, segnalazioni sonar e marker device) e avvia autonomamente sequenze di azioni (comandi al cargorobot, aggiornamenti display). Un POJO (Plain Old Java Object), componente passivo attivato da chiamate sincrone, non cattura questo comportamento.
- **Sistema event-driven:** sonar, marker device e cargorobot comunicano tramite eventi asincroni. Un modello a oggetti tradizionale gestisce l'asincronicità con difficoltà e produce codice artificioso.
- **Riduzione dell'abstraction gap:** il linguaggio QAK modella ogni entità del dominio come **attore** (automa a stati finiti, autonomo, message-driven), riducendo la distanza concettuale tra requisiti e codice. Il modello è **automaticamente eseguibile**, permettendo di verificare proprietà comportamentali già dallo Sprint 0.

3.4. Formalizzazione dei messaggi QAK

Request/Reply per interazioni con risposta attesa; **Dispatch** per notifiche asincrone senza risposta diretta.

3.4.1. customer / IOPort → cargoservice

```
Request load_request    : loadRequest(none)
Reply   load_accepted   : loadAccepted(slotID)    for load_request
Reply   load_retrylater : loadRetryLater(none)     for load_request
Reply   load_refused     : loadRefused(none)       for load_request
```

3.4.2. sonar → cargoservice

```
Dispatch sonar_data      : sonarData(distance)
Dispatch container_detected : containerDetected(none)
Dispatch sonar_failure    : sonarFailure(none)
```

3.4.3. cargoservice → cargorobot

```
Request  move_to_slot5 : moveToSlot5(none)
Reply    move_done      : moveDone(none)      for move_to_slot5

Request  move_to_slot   : moveToSlot(slotID)
Reply    move_done      : moveDone(none)      for move_to_slot
```

3.4.4. marker device → cargoservice

```
Dispatch marking_done : markingDone(containerID)
```

3.4.5. cargoservice → display / LED

```
Dispatch display_hold_state : holdState(state)
Dispatch display_status     : displayStatus(message)
Dispatch led_on              : ledOn(none)
Dispatch led_off             : ledOff(none)
```

3.5. Contesti logici

CONTESTO	COMPONENTI E RESPONSABILITÀ
ctxCargoService	Contiene l'attore cargoservice. Nucleo e punto di orchestrazione.
ctxIO	Interfacciamento con i dispositivi fisici: ioport, sonar, led, markerdevice. Gestisce l'abstraction gap hardware/software.
ctxRobot	Interfacciamento con il cargorobot DDR (controllo di movimento).

3.6. Schema della hold

Legenda: H = HOME S = sonar/sensor area 1-4 = slot1-4
 5 = slot5 I = IOPort X = ostacolo . = libero

	col0	col1	col2	col3	col4	col5	col6
riga0	[H]	[S]	[.]	[.]	[.]	[.]	[.]
riga1	[.]	[1]	[X]	[X]	[2]	[.]	[.]
riga2	[.]	[.]	[.]	[.]	[X]	[5]	[.]
riga3	[.]	[3]	[X]	[X]	[4]	[.]	[.]
riga4	[.]	[.]	[.]	[.]	[.]	[.]	[.]
riga5	[I]	[X]	[X]	[X]	[X]	[X]	[X]

Nota: La posizione esatta dell'IOPort e del sonar richiede conferma dalla committente (Domande D1 e D3). Lo schema è una prima interpretazione fedele alla figura allegata ai requisiti.

4. Test plan

I test funzionali verificano il comportamento del sistema rispetto ai requisiti, in modo indipendente dall'implementazione.

4.1. Accettazione della richiesta

Precondizioni: *disengaged*, non *Out of service*, IOPort libera, almeno uno slot libero.

Azioni: il customer preme il pushbutton.

Risultato: risposta *load_accepted* con slot riservato; sistema *engaged*; LED acceso.

4.2. Rifiuto (hold piena)

Precondizioni: *disengaged*, non *Out of service*, IOPort libera, slot1–slot4 tutti occupati.

Azioni: il customer preme il pushbutton.

Risultato: risposta *load_refused*; sistema *disengaged*; LED spento.

4.3. Retrylater (Out of service)

Precondizioni: sistema *Out of service*.

Azioni: il customer preme il pushbutton.

Risultato: risposta *load_retrylater*; stato immutato.

4.4. Retrylater (IOPort occupata)

Precondizioni: *disengaged*, non *Out of service*, IOPort occupata.

Azioni: il customer preme il pushbutton.

Risultato: risposta *load_retrylater*; sistema *disengaged*.

4.5. Timeout deposito container

Precondizioni: sistema *engaged*.

Azioni: il customer non deposita il container entro il tempo prefissato.

Risultato: sistema *disengaged*; LED spento.

4.6. Ciclo completo di carico

Precondizioni: *disengaged*, non *Out of service*, IOPort libera, slot disponibile.

Azioni: (a) pushbutton → (b) *load_accepted* → (c) deposito container → (d) rilevazione sonar → (e) cargorobot: IOPort→slot5 → (f) etichettatura → (g) cargorobot: slot5→slot riservato.

Risultato: container nello slot riservato; display aggiornato con “*Service working*”; sistema *disengaged*; LED spento.

4.7. Malfunzionamento sonar

Precondizioni: sistema operativo.

Azioni: sonar misura $D > D_{\text{FREE}}$ per ≥ 3 s consecutivi.

Risultato: sistema *Out of service*; display mostra “*Out of service*”.

4.8. Rilevazione container (sonar)

Precondizioni: sistema *engaged*; sensor area vuota.

Azioni: deposito container; sonar misura $D < D_{\text{FREE}}/2$ per ≥ 3 s.

Risultato: cargoservice riceve *container_detected* e avvia la movimentazione.

5. Project

Dall'analisi dei requisiti si propone una struttura a **tre sprint** con integrazione progressiva dei componenti: si parte dal nucleo logico con mock, si aggiungono gradualmente le interfacce software, e infine i dispositivi fisici reali.

5.1. Sprint 1: Core business

Goal: implementare cargoservice con tutti i dispositivi simulati (mock).

Funzionalità: ciclo di carico completo (RF1–RF10 con mock), struttura dati hold, LED simulato, timeout.

Test: TF01, TF02, TF05, TF06 (con mock).

5.2. Sprint 2: IOPort, display e sonar

Goal: interfaccia utente IOPort (pushbutton + display) e sonar software.

Funzionalità: ioport come interfaccia web, sonar simulato (rilevazione container e malfunzionamento), aggiornamento display con stato e messaggi.

Test: TF03, TF04, TF07, TF08.

5.3. Sprint 3: Dispositivi fisici

Goal: sostituire i mock con i dispositivi fisici reali e verificare end-to-end.

Funzionalità: cargorobot DDR reale, sonar fisico, marker device fisico, LED fisico.

Test: TF06 (end-to-end su hardware), TF07, TF08.

6. Team di lavoro

NOME E COGNOME	RUOLO
Davide Chirichella	Membro del gruppo
Gabriele Doti	Membro del gruppo
Daniele Maccagnan	Membro del gruppo